

Topological Sort

Ordinamento dei nodi di un DAG tale che
Se $(u, v) \in E$ allora nell'ordinamento
 $u < v$

1) Come verifico se un grafo orientato
è un DAG?

2) Se G è un DAG come determino
un ordinamento topologico di G ?

1) G è aciclico se in $\text{DFS}(G)$ non trovo
nessun arco all'indietro
 $\Downarrow$
da grigio a grigio

$\text{DFS}(G)$ 1

④ $(|V| + |E|)$ con
liste

- -

④ $(|V|^2)$ con matrici

}

DFS(G, u, TIME) {

;

for (each $v \in \text{Adj}[u]$) {

if (color[v] = Bianco) {

...

}

if (color[v] = grigio) {

print ("Grafo ciclico")

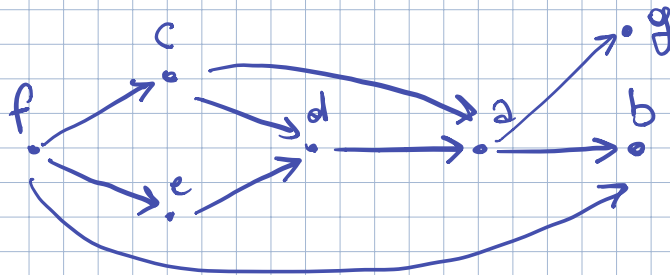
}

}

;

}

2)

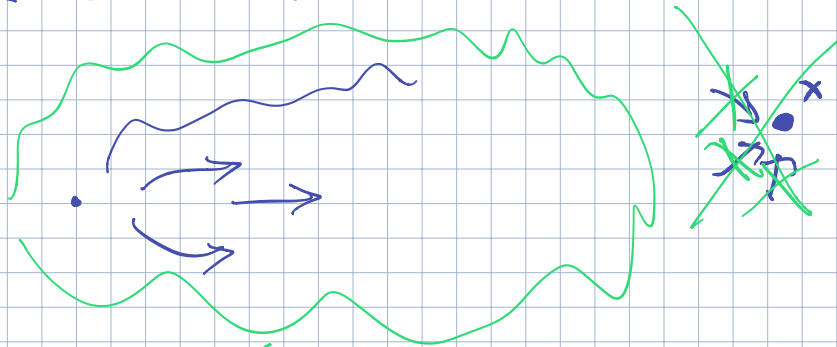


Chi può finire in fondo? g e b

Se
non hanno
archi uscenti.

Se G è aciclico esistono sicuramente
nodi senza archi uscenti? Sì

Ogni grafo aciclico ha almeno un nodo
senza archi uscenti e almeno un nodo
senza archi entranti



Resta un DAG

con $n-1$

nodi

$n = |V|$

Se

Posso procedere
nello stesso modo

x

↑

ultimo nel top. sort

← Cancellò x
e tutti gli
archi entranti
in x

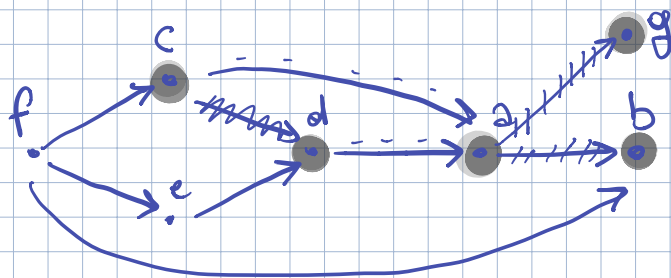
Con liste di adj

- x non ha archi uscenti $\rightarrow O(|V|)$
- eliminare x da ogni lista $\rightarrow O(|V| + |E|)$
- scrivo x in fondo al TS $\rightarrow O(1)$
- itero per $O(|V|)$ volte

$\downarrow$

$$O(|V| \cdot (|V| + |E|))$$

Riusciamo a fare meglio? Usando una sola visita DFS?



Sicuramente indipendentemente da dove parte la visita g oppure b è il primo nodo che diventa Nero

$\hookrightarrow$ c d a b g

Quando un nodo diventa nero sicuramente
tutti i nodi che lui raggiunge sono
già neri

Σ
Sono già stati inseriti
nelle liste di top-sort.

Algo Top. Sort

DFS in cui quando un nodo
diventa nero viene inserito in
testa a L lista

DFS-TS(G) { $\% G$ DAG

$L \leftarrow \emptyset$

DFS-visit(G, u, TIME, L)

return L

}

DFS-visit-TS (G, u, Time, L)

Insertisci_in_teste (L, u)

return L

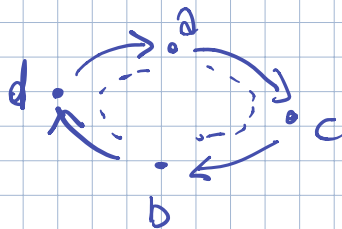
}

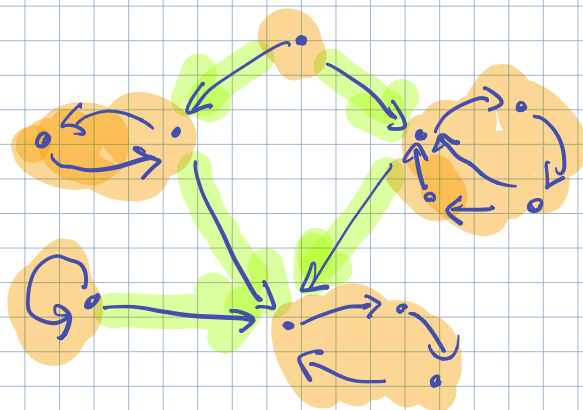
① ($|V| + |E|$)
con liste

② ($|V|^2$)
con matrici

~ ~

Come faccio nel caso di grafi orientati
ciclici?





Raggruppando
 i nodi nei
 cicli ottengo
 dei cluster
 di nodi
 $\sum$
 I cluster formano
 un DAG

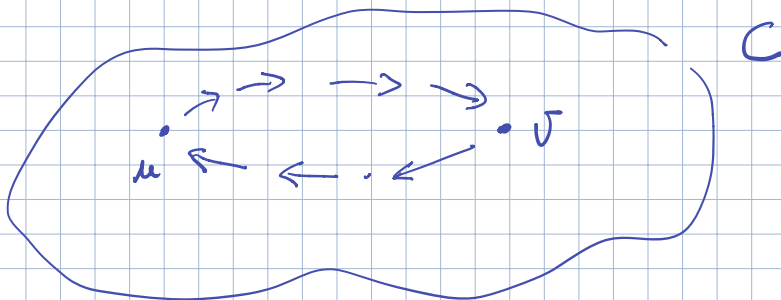
Come determinare

- i cluster $\mapsto$ Componenti Fort. connesse
- il DAG

Def.:

In G orientato una componente fortemente
 connessa è un insieme maximale
 di nodi mutualmente raggiungibili

$$C \subseteq V$$



Om.

Le componenti fortemente connesse
di G

$C_1 \quad C_2 \quad \dots \quad C_m$

sono una partizione di V

$$C_1 \cup C_2 \dots \cup C_m = V$$

$$C_i \neq \emptyset$$

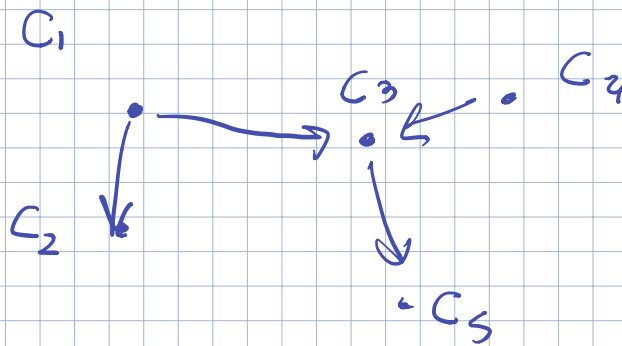
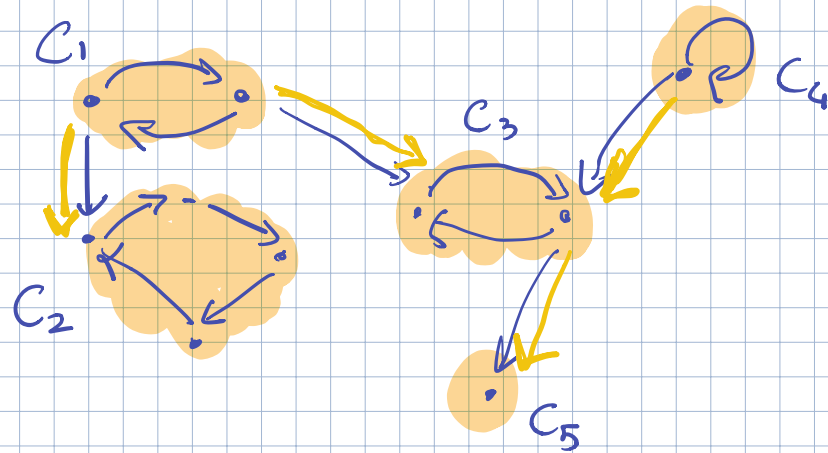
$$C_i \cap C_j = \emptyset \quad \text{con } i \neq j$$

Def G_{scc} ^{strongly conn. comp.} grafo delle comp. fort. conn.

$$G_{scc}(V_{scc}, E_{scc}) \text{ di } G=(V, E)$$

- V_{scc} e' insieme $C_1 \dots C_m$
delle c.f.c. (s.c.c.) di G
- $E_{scc} \quad (C_i, C_j) \in E_{scc} \text{ se}$
 $i \neq j$ e $\exists u \in C_i, v \in C_j$ t.c.
 $(u, v) \in E$

Esempio



Oss.

G_{sec} è sempre un DAG

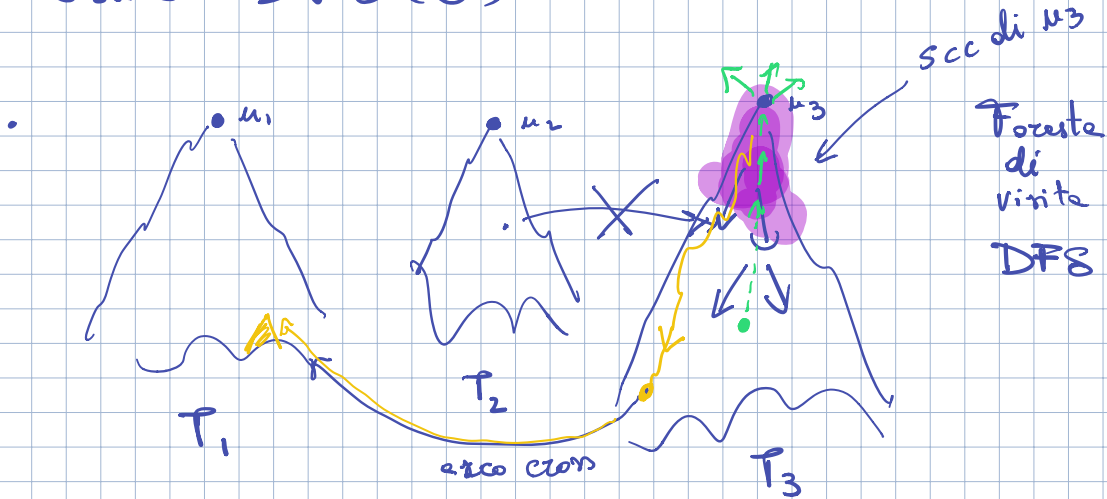
⇔

su G_{sec} posso usare top. sort

Come calcolo G_{sec} ? n^2 V_{sec}
Trovare le SCC

Idea

• Usare DFS (G)



Consideriamo u_3

Dove si trovano i nodi della s.c.c. di u_3 ?

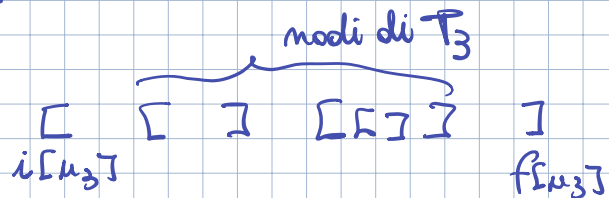
Devono stare in T_3

È tutto T_3 ? no

Che differenza c'è alla fine della visita tra u_3 e gli altri nodi di T_3 ?

$$\pi[u_3] = n+1$$

Fine visita u_3

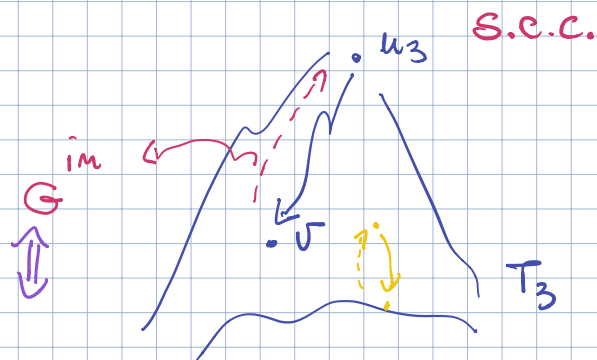


$$f[u_3] > f[v] \quad \forall v \in T_3$$

$$\forall v \in T_3$$

$$f[u_3] \text{ è } 2|V|$$

u_3 raggiunge i nodi che stanno in T_3
 tra i nodi di T_3 devo trovare quelli
 che raggiungono u_3



Se u_3

raggiunge v

in G^{-1} $\rightarrow$ inverti tutti gli archi)

Per calcolare $S.C.C.(u_3)$ eseguirò

$DFS(G^{-1}, u_3)$

Se eseguo
 $\forall v \in T_3$

$DFS_{vint}(G, v)$

e raggiungo u_3

$v \in S.C.C.(u_3)$